

Asteria Session Notes: Python OOP Fundamentals

1. Class

A class is a blueprint. RobotConfig describes what every configuration object should contain.

2. Object

`config = RobotConfig()` creates a new object from the RobotConfig blueprint.

3. `__init__`

A special method automatically called once when an object is created. It performs initialization, not booting.

4. `self`

`self` refers to the current object. `self.robot_name` stores data inside that specific object.

5. Instance Variables

Each object has its own copy of `self.robot_name`, `self.max_speed`, etc.

6. Config vs Robot

RobotConfig stores static specifications. Robot stores dynamic state and behavior.

7. Static vs Dynamic

Static: `robot_name`, `wheel_radius`, `battery_capacity`, `max_speed`. Dynamic: `battery_percentage`, `current_speed`, `current_position`, `current_mission`.

8. Ownership

Battery owns battery data. Localization owns position. Camera owns images. Ultrasonic owns obstacle distance. Motor Controller owns wheel speeds. Mission Manager owns mission.

9. Multiple Objects

Each `RobotConfig()` call creates a separate object. Editing `config1` does not affect `config2`.

10. First Asteria Code

```
class RobotConfig:
    def __init__(self):
        self.robot_name = 'Asteria'
        self.wheel_radius = 20
        self.max_speed = 50
        self.wheel_base = 30
```

11. Mental Models

Blueprint vs House, Passport vs Person, Factory Construction (`__init__`) vs Boot (`boot()`).

12. Next Steps

Finish RobotConfig, then build Robot with boot(), move_forward(), stop(), while learning OOP naturally through Asteria.